

Praktikum 7 — Classification Algorithms

Data Science • Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 7**. Assignment weight: **3%**. **This is the last meeting before the UTS**, which covers weeks 1–7.

What you will do

1. Compute information gain by hand, then let a tree do it.
2. Watch a tree overfit, and prune it properly.
3. Build naive Bayes and see why the "naive" assumption survives being false.
4. Measure distance concentration — the curse of dimensionality, numerically.
5. Tune an SVM's C and γ and see the boundary change.
6. Run a **fair** comparison of five models and disaggregate the errors.

The dataset. `data/credit.csv` — a genuinely imbalanced, genuinely non-trivial problem.

```
import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA = ROOT / "data"
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
                    "axes.spines.right": False, "axes.grid": True,
                    "grid.alpha": 0.3, "axes.titleweight": "bold"})
pd.set_option("display.width", 115)
```

1. Impurity and information gain, by hand (DEMO)

$$G = 1 - \sum_k p_k^2 \quad H = -\sum_k p_k \log_2 p_k$$

A node holds 100 customers, 30 of whom churned. A candidate split sends 40 to the left (24 churned) and 60 to the right (6 churned).

```
def gini(p):
    return 1 - (p ** 2 + (1 - p) ** 2)

def entropy(p):
    if p in (0, 1):
        return 0.0
    return -(p * np.log2(p) + (1 - p) * np.log2(1 - p))

parent, left, right = 0.30, 0.60, 0.10
nL, nR = 40, 60
n = nL + nR

print(f"{'':<10}{'parent':>10}{'left':>10}{'right':>10}{'weighted':>11}{'gain':>9}")
for name, f in (("Gini", gini), ("Entropy", entropy)):
    w = nL / n * f(left) + nR / n * f(right)
    print(f"{name:<10}{f(parent):>10.4f}{f(left):>10.4f}{f(right):>10.4f}"
          f"{w:>11.4f}{f(parent) - w:>9.4f}")

print("\nNote the LEFT child is MORE impure than the parent (0.480 > 0.420).")
print("That is not a failure: what matters is the weighted average across")
print("both children. A split is judged by the partition, never by one side.")
```

```
# The two criteria almost always agree on which split to take.
fig, ax = plt.subplots(figsize=(5, 3.2))
ps = np.linspace(0.001, 0.999, 400)
ax.plot(ps, [gini(p) for p in ps], label="Gini", color="#2E6F9E", lw=2)
ax.plot(ps, [entropy(p) for p in ps], label="Entropy (bits)",
        color="#9C3535", lw=2)
ax.plot(ps, [2 * gini(p) for p in ps], label="2 x Gini (rescaled)",
        color="#2E6F9E", lw=1, ls=":")
ax.set(xlabel="proportion of class 1", ylabel="impurity",
      title="Gini and entropy have the same shape")
ax.legend(fontsize=8, frameon=False)
plt.show()
```

TASK 7.1

Report the **Gini information gain** of that split.

```
information_gain = None

checks.task("7.1", information_gain)
```

2. Trees overfit, and how to stop them (DEMO)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import roc_auc_score, accuracy_score, confusion_matrix

credit = pd.read_csv(DATA / "credit.csv")
y = (credit["default"] == "yes").astype(int)

X = credit.drop(columns=["default"]).copy()
X["dti"] = X["debt"] / X["income"].clip(lower=1)
X["pti"] = X["monthly_payment"] / (X["income"] / 12).clip(lower=1)
X["util"] = X["balance"] / X["credit_limit"].clip(lower=1)
X = pd.get_dummies(X, drop_first=False).astype(float)
X = X.fillna(X.median())

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y,
                                     random_state=42)
print(f"train {Xtr.shape}   test {Xte.shape}   default rate {y.mean():.1%}")
```

```
rows = []
for depth in [1, 2, 3, 4, 5, 6, 8, 10, 15, 20, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(Xtr, ytr)
    rows.append({
        "max_depth": depth if depth else "None",
        "leaves": t.get_n_leaves(),
        "train_auc": roc_auc_score(ytr, t.predict_proba(Xtr)[: , 1]),
        "test_auc": roc_auc_score(yte, t.predict_proba(Xte)[: , 1]),
    })
overfit = pd.DataFrame(rows)
overfit["gap"] = overfit["train_auc"] - overfit["test_auc"]
print(overfit.round(4).to_string(index=False))
```

```
fig, ax = plt.subplots(figsize=(6, 3.6))
xs = range(len(overfit))
ax.plot(xs, overfit["train_auc"], marker="o", ms=4, label="train",
        color="#9C3535")
ax.plot(xs, overfit["test_auc"], marker="s", ms=4, label="test",
        color="#2E6F9E")
ax.set_xticks(xs, overfit["max_depth"], rotation=45)
ax.set_xlabel="max_depth", ylabel="ROC-AUC",
        title="An unconstrained tree memorises the training data")
best = overfit["test_auc"].idxmax()
ax.axvline(best, color="#356B3F", ls="--", lw=1,
           label=f"best test AUC (depth {overfit.loc[best, 'max_depth']})")
ax.legend(fontsize=8, frameon=False)
plt.show()

print(f"Unconstrained tree: {overfit.iloc[-1]['leaves']:.0f} leaves, "
      f"train AUC {overfit.iloc[-1]['train_auc']:.3f}, "
      f"test AUC {overfit.iloc[-1]['test_auc']:.3f}")
print("It has memorised the training set and learned little that transfers.")
```

Cost-complexity pruning

More principled than guessing `max_depth`: grow the full tree, then minimise $\$R_{\alpha}(T) = R(T) + \alpha|T|$, trading misclassification against size.

```

path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(Xtr, ytr)
alphas = path.ccp_alphas[:-1][::max(1, len(path.ccp_alphas) // 40)]

prune_rows = []
for a in alphas:
    t = DecisionTreeClassifier(random_state=42, ccp_alpha=a).fit(Xtr, ytr)
    prune_rows.append({
        "ccp_alpha": a, "leaves": t.get_n_leaves(),
        "train_auc": roc_auc_score(ytr, t.predict_proba(Xtr)[: , 1]),
        "test_auc": roc_auc_score(yte, t.predict_proba(Xte)[: , 1]),
    })
pruned = pd.DataFrame(prune_rows)
best_row = pruned.loc[pruned["test_auc"].idxmax()]
print(f"best ccp_alpha = {best_row['ccp_alpha']:.6f} "
      f"({best_row['leaves']:.0f} leaves, test AUC {best_row['test_auc']:.4f})")

fig, ax = plt.subplots(figsize=(6, 3.4))
ax.plot(pruned["ccp_alpha"], pruned["train_auc"], label="train",
        color="#9C3535")
ax.plot(pruned["ccp_alpha"], pruned["test_auc"], label="test", color="#2E6F9E")
ax.axvline(best_row["ccp_alpha"], color="#356B3F", ls="--", lw=1)
ax.set(xscale="log", xlabel=r"$\alpha$ (pruning strength)", ylabel="ROC-AUC",
        title="Cost-complexity pruning traces the bias-variance curve")
ax.legend(fontsize=8, frameon=False)
plt.show()

```

```

# A shallow tree can be read aloud to a stakeholder. That is genuinely rare.
small = DecisionTreeClassifier(max_depth=3, min_samples_leaf=40,
                              random_state=42).fit(Xtr, ytr)
fig, ax = plt.subplots(figsize=(15, 6))
plot_tree(small, feature_names=list(X.columns), class_names=["no", "yes"],
          filled=True, rounded=True, fontsize=7, impurity=False, ax=ax)
ax.set_title("A depth-3 tree: readable, auditable, and defensible to a regulator")
plt.show()

```

3. Naive Bayes (DEMO)

$$\hat{y} = \arg\max_k \left[\log P(y=k) + \sum_j \log P(x_j \mid y=k) \right]$$

The **naive** assumption is conditional independence given the class. It is essentially never true — and the classifier works anyway.

```

from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

nb = GaussianNB().fit(Xtr, ytr)
nb_proba = nb.predict_proba(Xte)[: , 1]
print(f"Gaussian NB: accuracy {accuracy_score(yte, nb.predict(Xte)):.4f}, "
      f"ROC-AUC {roc_auc_score(yte, nb_proba):.4f}")

# How badly is the independence assumption violated?
corr = Xtr[["income", "debt", "loan_amount", "monthly_payment",
           "credit_limit", "balance"]].corr()
print("\nPairwise correlations among predictors (independence assumes ~0):")
print(corr.round(3).to_string())

```

Several correlations exceed 0.5 — `monthly_payment` is a deterministic function of `loan_amount` and `term_months`. The assumption is violated flagrantly, and the classifier still ranks usefully.

The reason: classification needs only the correct **ranking** of the posteriors, not their correct values. Dependence distorts the magnitudes badly while often leaving the argmax intact.

The consequence is that its **probabilities** are unusable without calibration:

```
from sklearn.linear_model import LogisticRegression

lr_cmp = make_pipeline(StandardScaler(),
                        LogisticRegression(max_iter=3000)).fit(Xtr, ytr)
lr_proba = lr_cmp.predict_proba(Xte)[: , 1]

fig, axes = plt.subplots(1, 2, figsize=(11, 3.4))
for ax, pr, name in [(axes[0], nb_proba, "Gaussian naive Bayes"),
                     (axes[1], lr_proba, "Logistic regression")]:
    ax.hist(pr, bins=40, color="#2E6F9E", edgecolor="white", linewidth=0.3)
    ax.set(title=f"{name}\npredicted probabilities", xlabel="P(default)",
           ylabel="log")
plt.show()

print(f"NB : {(nb_proba < 0.01).mean():.1%} of predictions below 0.01, "
      f"{(nb_proba > 0.99).mean():.1%} above 0.99")
print(f"LR : {(lr_proba < 0.01).mean():.1%} below 0.01, "
      f"{(lr_proba > 0.99).mean():.1%} above 0.99")
print("\nNaive Bayes piles its mass at the extremes -- it counts correlated")
print("features as independent evidence, so it is wildly over-confident.")
print("Use its LABEL; calibrate before using its PROBABILITY.")
```

Laplace smoothing

If a feature value never co-occurs with class k in training, $P(x_j \mid y=k) = 0$ and the product for that class becomes zero regardless of all other evidence. One unseen word vetoes the entire document.

```
V, N_spam, alpha = 50_000, 20_000, 1

print("A word that never appeared in any spam message:")
print(f" unsmoothed P(word | spam) = {0 / N_spam:.6f}    <- vetoes everything")
print(f" Laplace    P(word | spam) = "
      f"{(0 + alpha) / (N_spam + alpha * V):.8f}    <- small, but not zero")
print("\nNever set alpha=0 on text data.")
```

4. The curse of dimensionality (DEMO)

In high dimensions, distances **concentrate**: the ratio between the nearest and farthest neighbour tends to 1, so "nearest" stops meaning "similar".

```

rng = np.random.default_rng(0)
rows_c = []
for p in (2, 10, 100, 1000):
    Xr = rng.uniform(size=(1000, p))
    q = rng.uniform(size=p)
    d = np.linalg.norm(Xr - q, axis=1)
    rows_c.append({"p": p, "nearest": d.min(), "farthest": d.max(),
                  "ratio": d.max() / d.min(),
                  "relative_contrast": (d.max() - d.min()) / d.min()})
conc = pd.DataFrame(rows_c)
print(conc.round(3).to_string(index=False))

fig, ax = plt.subplots(figsize=(5.5, 3.4))
for p, colour in zip((2, 10, 100, 1000),
                    ["#173F5F", "#2E6F9E", "#A97B21", "#9C3535"]):
    Xr = rng.uniform(size=(1000, p))
    q = rng.uniform(size=p)
    d = np.linalg.norm(Xr - q, axis=1)
    ax.hist(d / d.mean(), bins=50, histtype="step", lw=1.6, density=True,
           color=colour, label=f"p = {p}")
ax.set(xlabel="distance / mean distance", ylabel="density",
      title="Distances concentrate as dimension grows")
ax.legend(fontsize=8, frameon=False)
plt.show()

print("\nAt p=1000 every point sits at essentially the same distance from the")
print("query. The neighbourhood is arbitrary and the k-NN vote is near random.")
print("This affects EVERY distance-based method: k-NN, k-means, RBF-SVM.")

```

TASK 7.2

Report the farthest/nearest **ratio** at $p = 1000$.

```

rasio_jarak = None

checks.task("7.2", rasio_jarak)

```

5. Support vector machines (DEMO)

SVM maximises the **margin**. The soft-margin formulation introduces C as an inverse regularisation strength; the RBF kernel's γ controls the reach of a single training example.

```

from sklearn.svm import SVC
from sklearn.datasets import make_moons

Xm, ym = make_moons(n_samples=300, noise=0.28, random_state=0)
Xm = StandardScaler().fit_transform(Xm)

fig, axes = plt.subplots(2, 3, figsize=(13, 7))
xx, yy = np.meshgrid(np.linspace(-2.5, 2.5, 250), np.linspace(-2.5, 2.5, 250))
grid = np.c_[xx.ravel(), yy.ravel()]

for i, C in enumerate([0.1, 1, 100]):
    for j, gamma in enumerate([0.1, 10]):
        ax = axes[j, i]
        svm = SVC(kernel="rbf", C=C, gamma=gamma).fit(Xm, ym)
        Z = svm.decision_function(grid).reshape(xx.shape)
        ax.contourf(xx, yy, Z > 0, alpha=0.18, cmap="coolwarm")
        ax.contour(xx, yy, Z, levels=[-1, 0, 1], colors="k",
                    linestyle=["--", "-", "--"], linewidths=[0.7, 1.3, 0.7])
        ax.scatter(Xm[:, 0], Xm[:, 1], c=ym, cmap="coolwarm", s=12,
                    edgecolor="k", linewidth=0.2)
        ax.scatter(svm.support_vectors_[:, 0], svm.support_vectors_[:, 1],
                    s=60, facecolors="none", edgecolors="#356B3F", linewidth=0.8)
        ax.set(title=f"C={C}, gamma={gamma} ({len(svm.support_vectors_)} SVs)",
                xticks=[], yticks=[])
        ax.title.set_fontsize(9)
fig.suptitle("Circled points are support vectors – they alone define the boundary",
             y=1.01, fontsize=10)
plt.show()

```

Reading the grid:

- **Large C** penalises violations heavily → narrow margin, fits the training data closely → low bias, high variance.
- **Large γ** means a narrow kernel → influence is local, the boundary becomes wiggly → overfitting. In the limit each support vector claims only its own neighbourhood.
- **Small γ** → wide kernel, boundary approaching linear.

C and γ **interact**, so tune them jointly on a log grid.

```

from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    make_pipeline(StandardScaler(), SVC(kernel="rbf")),
    {"svc__C": np.logspace(-1, 3, 5), "svc__gamma": np.logspace(-4, 0, 5)},
    cv=3, scoring="roc_auc")
grid_search.fit(Xtr, ytr)
print("best params:", grid_search.best_params_)
print(f"best CV ROC-AUC: {grid_search.best_score_:.4f}")

heat = (pd.DataFrame(grid_search.cv_results_)
        .pivot_table(index="param_svc__gamma", columns="param_svc__C",
                      values="mean_test_score"))
fig, ax = plt.subplots(figsize=(5.2, 3.8))
im = ax.imshow(heat.to_numpy(), cmap="viridis", aspect="auto")
ax.set_xticks(range(len(heat.columns)), [f"{c:g}" for c in heat.columns])
ax.set_yticks(range(len(heat.index)), [f"{g:g}" for g in heat.index])
ax.set(xlabel="C", ylabel="gamma", title="CV ROC-AUC over the (C, gamma) grid")
for i in range(heat.shape[0]):
    for j in range(heat.shape[1]):
        ax.text(j, i, f"{heat.iloc[i, j]:.3f}", ha="center", va="center",
                fontsize=7, color="white")
ax.grid(False)
fig.colorbar(im, ax=ax, shrink=0.85)
plt.show()

```

6. A fair comparison (DEMO)

Identical folds, identical preprocessing, more than one metric, and the errors **disaggregated**. Two models with identical accuracy can make entirely different mistakes.

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_validate

models = {
    "LogReg": make_pipeline(StandardScaler(),
                           LogisticRegression(C=1.0, max_iter=5000)),
    "Tree": DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                                   random_state=42),
    "GaussNB": GaussianNB(),
    "kNN": make_pipeline(StandardScaler(),
                        KNeighborsClassifier(n_neighbors=25,
                                           weights="distance")),
    "SVM-RBF": make_pipeline(StandardScaler(),
                             SVC(C=10, gamma="scale", probability=True,
                                 random_state=42)),
}

cv = StratifiedKFold(5, shuffle=True, random_state=42)
rows_m = []
for name, m in models.items():
    cvres = cross_validate(m, Xtr, ytr, cv=cv,
                          scoring=["accuracy", "roc_auc"])

    m.fit(Xtr, ytr)
    pr = m.predict_proba(Xte)[: , 1]
    tn, fp, fn, tp = confusion_matrix(yte, m.predict(Xte)).ravel()
    rows_m.append({
        "model": name,
        "cv_acc": cvres["test_accuracy"].mean(),
        "cv_auc": cvres["test_roc_auc"].mean(),
        "cv_auc_sd": cvres["test_roc_auc"].std(),
        "test_auc": roc_auc_score(yte, pr),
        "missed_default": fn,      # the expensive error
        "false_alarm": fp,
    })

comparison = pd.DataFrame(rows_m).set_index("model")
print(comparison.round(4).to_string())
print(f"\nbaseline: always predict 'no default' -> accuracy "
      f"{1 - ytr.mean():.4f}")

```

```

from sklearn.metrics import roc_curve

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for name, m in models.items():
    pr = m.predict_proba(Xte)[: , 1]
    fpr, tpr, _ = roc_curve(yte, pr)
    axes[0].plot(fpr, tpr, lw=1.4,
                 label=f"{name} ({roc_auc_score(yte, pr):.3f})")
axes[0].plot([0, 1], [0, 1], ls=":", color="grey")
axes[0].set(xlabel="false positive rate", ylabel="true positive rate",
            title="ROC curves on the held-out set")
axes[0].legend(fontsize=8, frameon=False)

idx = np.arange(len(comparison))
axes[1].barh(idx - 0.2, comparison["missed_default"], height=0.38,
             color="#9C3535", label="missed defaults (costly)")
axes[1].barh(idx + 0.2, comparison["false_alarm"], height=0.38,
             color="#2E6F9E", label="false alarms (cheap)")
axes[1].set_yticks(idx, comparison.index)
axes[1].set(xlabel="count", title="The errors, disaggregated")
axes[1].legend(fontsize=8, frameon=False)
axes[1].grid(axis="y", visible=False)
plt.show()

```

Read the errors, not just the score. Models with near-identical accuracy make very different mistakes, and in a lending context a missed default and a needless rejection have wildly different costs. Which model you deploy depends on that ratio — which is a business decision, not a statistical one.

TASK 7.3

Report the best held-out ROC-AUC across the five models.

```

auc_terbaik = None

checks.task("7.3", auc_terbaik)

```

The pitfall in this very table

The tree and k -NN hyperparameters above were chosen by hand; the others were left near default. That makes this a comparison of *"performance at defaults"* — a weaker claim than *"performance of the model family"*. Either tune all of them with the same budget, or say plainly which claim you are making. Task E below asks you to fix it.

7. Take-home (assessed — and the best UTS preparation available)

A. Information gain by hand. A node has 200 observations: 150 class A, 50 class B. Compute Gini and entropy. Then evaluate a split producing (120 A, 10 B) and (30 A, 40 B): compute the information gain under both criteria and state whether the split is worth making.

B. The diagonal boundary. Explain why a decision tree needs many splits to approximate $x_1 = x_2$, and sketch the resulting partition. Then name **one** feature you could engineer that would let a depth-1 tree solve it exactly. Verify your answer in code.

C. Concentration and PCA. Reproduce section 4, then repeat it with data lying on a 2-dimensional linear subspace embedded in p dimensions (generate 2 coordinates, pad with near-zero noise). Explain why the ratio behaves differently, and what this implies about applying PCA before k -NN.

D. k-NN sensitivity. State the effect on bias and on variance of: (a) increasing k ; (b) `weights="distance"` instead of `"uniform"`; (c) adding 50 pure-noise features; (d) failing to standardise a feature measured in millions. Verify (c) and (d) empirically on the credit data.

E. A genuinely fair comparison. Redo section 6 with `GridSearchCV` around **every** model, using the same CV object, the same scoring metric and a comparable budget. Report whether the ranking changes, and discuss what that says about the pitfall above.

F. Deploy a decision. Using the cost formula from week 6, choose an operating threshold for your best model under the assumption that a missed default costs **8×** a needless rejection. Report the expected cost per application rather than accuracy, and write no more than 300 words on which model you would deploy — including **at least one reason that is not about the score.**

Selamat belajar untuk UTS!

The mid-term covers weeks 1–7. The most valuable revision is not re-reading — it is re-running these seven notebooks from a fresh kernel and making sure you can explain **why** each cell does what it does.